



University of Piraeus

School of Information and Telecommunications Technologies

Department of Digital Systems

Level: Postgraduate Program of Study

Network Security

Case 7

Supervisor: Prof. Christos Xenakis

Kalaitzidis Ioannis

ioannis_kalaitzidis@ssl-unipi.gr

Piraeus

24/12/2025

Abstract

Implementation of "Defense-in-Depth" security architecture in an Ubuntu Server environment, with the combined use of iptables (Network Firewall) for port control and ModSecurity (WAF) for the protection of web applications. Implementation of a strict security policy with exclusive allowed access to SSH, HTTP/S and port 9001, and universal rejection (DROP) of all other traffic. Confirm effectiveness through checks by Kali Linux, successfully block unauthorized connections from the firewall and prevent SQL Injection attacks from the WAF.

Contents

Introduction.....	1
Practical piece of work.....	2
Bibliography	10

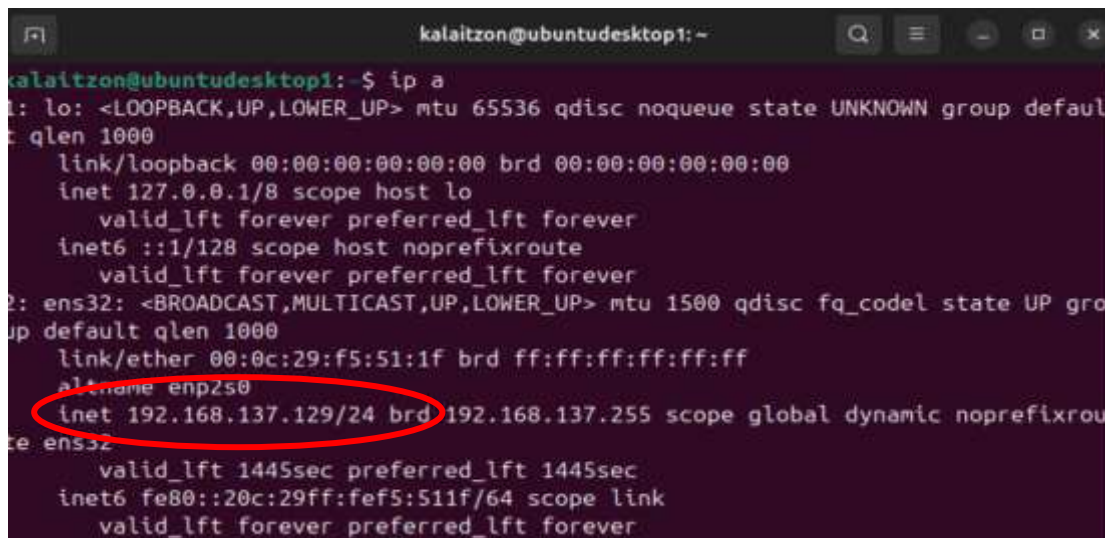
Introduction

In the context of this laboratory exercise, the implementation and configuration of a security architecture based on the "Defense-in-Depth" model was carried out. For the needs of the scenario, two virtual machines were used with distinct roles, simulating an attack and defense environment.

The first system (Ubuntu) operates hybridly as a server and security node, implementing host-based security measures. As a Server, it hosts the necessary services, namely the Apache Web Server for HTTP/HTTPS traffic management, as well as a custom service on port 9001. Unlike infrastructures that rely exclusively on perimeter firewalls, this implementation activates security directly on the Server through two levels of control: iptables (Network Firewall) for network packet filtering and ModSecurity (Web Application Firewall-WAF) for detecting malicious data within applications.

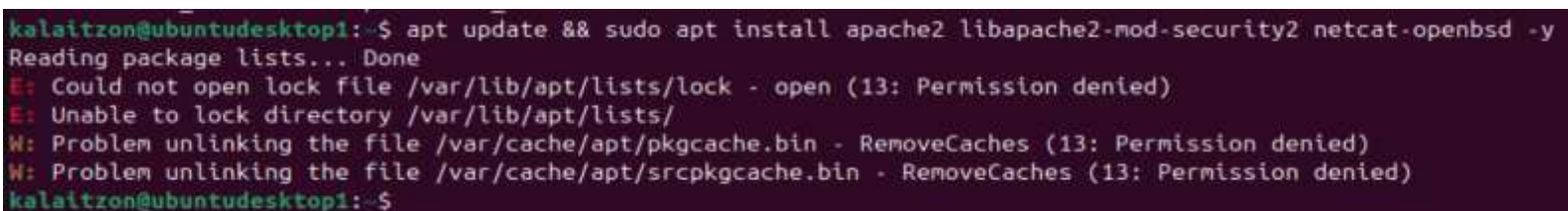
The second system (Kali Linux) assumes the role of Client/Attacker. Through it, connectivity checks and attack simulations are conducted, with the aim of verifying the integrity and effectiveness of the filtering rules applied to the Ubuntu VM.

Practical piece of work



```
kalaitzon@ubuntudesktop1: ~  
kalaitzon@ubuntudesktop1:~$ ip a  
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default  
    qlen 1000  
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00  
    inet 127.0.0.1/8 scope host lo  
        valid_lft forever preferred_lft forever  
    inet6 ::1/128 scope host noprefixroute  
        valid_lft forever preferred_lft forever  
2: ens32: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP gro  
up default qlen 1000  
    link/ether 00:0c:29:f5:51:1f brd ff:ff:ff:ff:ff:ff  
    altname enp250  
    inet 192.168.137.129/24 brd 192.168.137.255 scope global dynamic noprefixrou  
te ens32  
        valid_lft 1445sec preferred_lft 1445sec  
    inet6 fe80::20c:29ff:fef5:511f/64 scope link  
        valid_lft forever preferred_lft forever
```

First we need to find the ip of Ubuntu with the command **ip a**. The ip is:
192.168.137.129



```
kalaitzon@ubuntudesktop1:~$ apt update && sudo apt install apache2 libapache2-mod-security2 netcat-openbsd -y  
Reading package lists... Done  
E: Could not open lock file /var/lib/apt/lists/lock - open (13: Permission denied)  
E: Unable to lock directory /var/lib/apt/lists/  
W: Problem unlinking the file /var/cache/apt/pkgcache.bin - RemoveCaches (13: Permission denied)  
W: Problem unlinking the file /var/cache/apt/srcpkgcache.bin - RemoveCaches (13: Permission denied)  
kalaitzon@ubuntudesktop1:~$
```

We will then download and update the following applications:

- **Apache2:** It is the most popular web server and is required to have HTTP/HTTPS functionality. It is the server to which we will create the website to which the client will connect (Kali Linux).
- **Libapache2-mod-security2:** It is the web application firewall (WAF). It offers security to the server from web attacks. It will essentially go between Apache and control the data to block attacks such as SQL injections etc.
- **Netcat-openbsd:** It is the tool we will use to read and write data to network connections. Because the main requirement for the exercise to be open is that port 9001 is open, we will use netcat to open this door.

In the screenshot we notice that this command that installs and updates the above applications does not work because we did not have administrative rights. With the addition of "**sudo**" in front of the remaining command, the problem is solved.

Then we activate the web server with the command "**sudo service**

```
kalaitzon@ubuntudesktop1:~$ sudo service apache2 start
kalaitzon@ubuntudesktop1:~$ sudo service apache2 status
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
   Active: active (running) since Wed 2025-12-24 00:55:14 EET; 1min 49s ago
     Docs: https://httpd.apache.org/docs/2.4/
   Process: 6385 ExecStart=/usr/sbin/apachectl start (code=exited, status=0/SUCCESS)
    Main PID: 6389 (apache2)
       Tasks: 55 (limit: 9374)
      Memory: 29.8M (peak: 29.6M)
         CPU: 499ms
        CGroup: /system.slice/apache2.service
                └─6389 /usr/sbin/apache2 -k start
                  └─6394 /usr/sbin/apache2 -k start
                    └─6396 /usr/sbin/apache2 -k start

Dec 24 00:55:13 ubuntudesktop1.kalaitzon systemd[1]: Starting apache2.service - The Apache HTTP Server...
Dec 24 00:55:14 ubuntudesktop1.kalaitzon systemd[1]: Started apache2.service - The Apache HTTP Server.
kalaitzon@ubuntudesktop1:~$
```

apache2 start" and to check if the apache server is working, we use the command "**sudo service apache2 status**". The result, as shown in the screenshot, shows that everything went well and that the server is open and waiting for HTTP/HTTPS connections.

```
kalaitzon@ubuntudesktop1:~$ nc -l -p 9001
```

Here we will use the netcat tool and with the command `nc -l -p 9001` we will create a web service. With the parameter "-l" we tell the system to wait for a connection (listen) instead of connecting somewhere else. With the parameter "-p" we indicate which door to open, i.e. 9001. Without this command, even if we set the firewall to allow 9001, the door would be closed because there would be no program to use it. As soon as we run the command, the cursor will flash and it means that the command is running normally and the server is in standby mode. We will leave this terminal open so that the door continues to "listen" and we will open a new terminal.

```

kalaitzon@ubuntuDesktop1:~$ sudo iptables -F
[sudo] password for kalaitzon:
kalaitzon@ubuntuDesktop1:~$ sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT
kalaitzon@ubuntuDesktop1:~$ sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT
sudo: iptables: command not found
es: command not found
kalaitzon@ubuntuDesktop1:~$ sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT
kalaitzon@ubuntuDesktop1:~$ sudo iptables -A INPUT -p tcp --dport 443 -j ACCEPT
kalaitzon@ubuntuDesktop1:~$ sudo iptables -A INPUT -p tcp --dport 9001 -j ACCEPT
kalaitzon@ubuntuDesktop1:~$ sudo iptables -A INPUT -i lo -j ACCEPT
sudo: iptables: command not found
es: command not found
kalaitzon@ubuntuDesktop1:~$ sudo iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
kalaitzon@ubuntuDesktop1:~$ sudo iptables -P INPUT DROP
kalaitzon@ubuntuDesktop1:~$ sudo iptables -P FORWARD DROP
kalaitzon@ubuntuDesktop1:~$ sudo iptables -L -v -n
Chain INPUT (policy DROP 0 packets, 0 bytes)
pkts bytes target      prot opt in     out    source            destination
 0     0 ACCEPT     6     -- *     *       0.0.0.0/0         0.0.0.0/0         tcp dpt:22
 0     0 ACCEPT     6     -- *     *       0.0.0.0/0         0.0.0.0/0         tcp dpt:80
 0     0 ACCEPT     6     -- *     *       0.0.0.0/0         0.0.0.0/0         tcp dpt:443
 0     0 ACCEPT     6     -- *     *       0.0.0.0/0         0.0.0.0/0         tcp dpt:9001
 0     0 ACCEPT     6     -- lo    *       0.0.0.0/0         0.0.0.0/0
 0     0 ACCEPT     6     -- *     *       0.0.0.0/0         0.0.0.0/0         ctstate RELATED,ESTABLISHED

Chain FORWARD (policy DROP 0 packets, 0 bytes)
pkts bytes target      prot opt in     out    source            destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
pkts bytes target      prot opt in     out    source            destination
kalaitzon@ubuntuDesktop1:~$

```

Next we will create the firewall rules through the iptables tool. Essentially, with the firewall we will determine who is allowed to enter and who is not.

With the command "**sudo iptables -F**" we delete (-F) all the previous rules. We do this to have a "clean" basis so that no problem is created by the pre-existing ones.

With the command "**sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT**", we add a rule for the TCP protocol to port 22, with a command to accept (ACCEPT). We open the door for SSH so that it is possible to connect remotely to the server.

With the command "**sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT**" and the command "**sudo iptables -A INPUT -p tcp --dport 443 -j ACCEPT**" we allow movement on doors 80 and 443. Essentially we open the access for the web server (HTTP/HTTPS).

With the command "**sudo iptables -A INPUT -p tcp --dport 9001 -j ACCEPT**" we allow movement to door 9001. We did this because it is a specific requirement of Use Case 7, in order to allow access to the custom service hosted by the server to serve the client.

With the command "**sudo iptables -A INPUT -i lo -j ACCEPT**" we allow the movement in the loopback interface (lo). In other words, we allow the server to talk to itself. We do this because many system services communicate with each other via localhost (127.0.0.1) and blocking this traffic would cause the operating system to malfunction.

With the command "**sudo iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT**" we allow packets that belong to already open connections. Essentially, we ensure that if the server requests

something from the internet, the response will be able to return. Without this, the firewall would cut off the responses to its own requests.

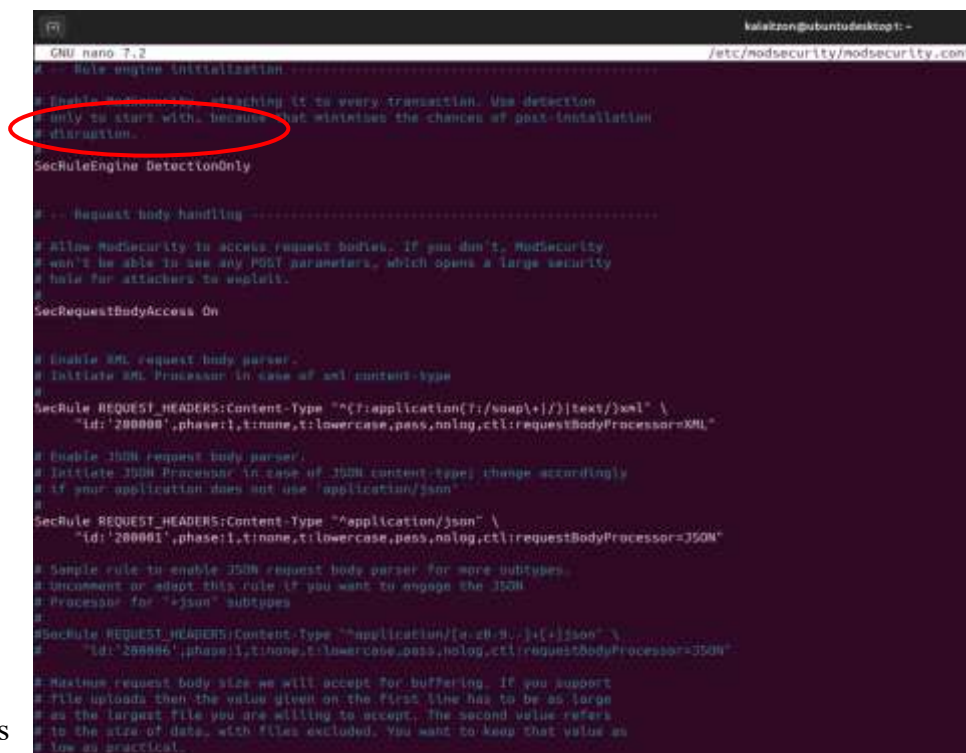
With the command "**sudo iptables -P INPUT DROP**" and the command "**sudo iptables -P FORWARD DROP**" we change the policy (-P) to DROP. This is the most important part of the commands we wrote as if a package does not match any of the above rules (22, 80, 443, 9001), then ignore it. That is, we implement here a situation in which all ports are closed except the mentioned ones.

Finally, with the command "**sudo iptables -L -v -n**" we displayed the list of active rules for confirmation. The list confirms what we did above (the doors we want are ACCEPT and the policy in general is DROP).

Then with the command "**sudo cp /etc/modsecurity/modsecurity.conf-**

```
kalaitzon@ubuntudesktop1:~$ sudo cp /etc/modsecurity/modsecurity.conf-recommended /etc/modsecurity/modsecurity.conf
kalaitzon@ubuntudesktop1:~$ sudo nano /etc/modsecurity/modsecurity.conf
kalaitzon@ubuntudesktop1:~$ sudo service apache2 restart
```

recommended /etc/modsecurity/modsecurity.conf" we copy the file with the recommended settings and create a master configuration file (modsecurity.conf). We do this because modsecurity does not have an active configuration file. We need to create this file to start working properly. Then with the command "**sudo nano /etc/modsecurity/modsecurity.conf**" open the specific file and change the setting from "**SecRuleEngine DetectionOnly**" to "**SecRuleEngine On**" as shown in the image below.



```
GNU nano 2.2 /etc/modsecurity/modsecurity.conf
# Rule engine initialization
# Enable 'SecRuleEngine' attaching it to every transaction. Use detection
# only to start with, because that minimizes the chances of post-installation
# disruption.
SecRuleEngine DetectionOnly

# -- Request body handling -----
# Allow ModSecurity to access request bodies. If you don't, ModSecurity
# won't be able to see any POST parameters, which opens a large security
# hole for attackers to exploit.
SecRequestBodyAccess On

# Enable XML request body parser.
# Initiate XML Processor in case of xml content-type
SecRule REQUEST_HEADERS:Content-Type "(?:application/(?:soap|.+)/|text/xml) \
    "id:'200000',phase:1,t:none,t:lowercase,pass,nolog,ctl:requestBodyProcessor=XML"

# Enable JSON request body parser.
# Initiate JSON Processor in case of JSON content-type; change accordingly
# if your application does not use 'application/json'
SecRule REQUEST_HEADERS:Content-Type "application/json" \
    "id:'200001',phase:1,t:none,t:lowercase,pass,nolog,ctl:requestBodyProcessor=JSON"

# Sample rule to enable JSON request body parser for more subtypes.
# Uncomment or adapt this rule if you want to engage the JSON
# Processor for "+json" subtypes
SecRule REQUEST_HEADERS:Content-Type "application/(?:x-nd+|.+)/json" \
    "id:'200002',phase:1,t:none,t:lowercase,pass,nolog,ctl:requestBodyProcessor=JSON"

# Maximum request body size we will accept for buffering. If you support
# file uploads then the value given on the first line has to be as large
# as the largest file you are willing to accept. The second value refers
# to the size of data, with files excluded. You want to keep that value as
# low as practical.
```

This

was

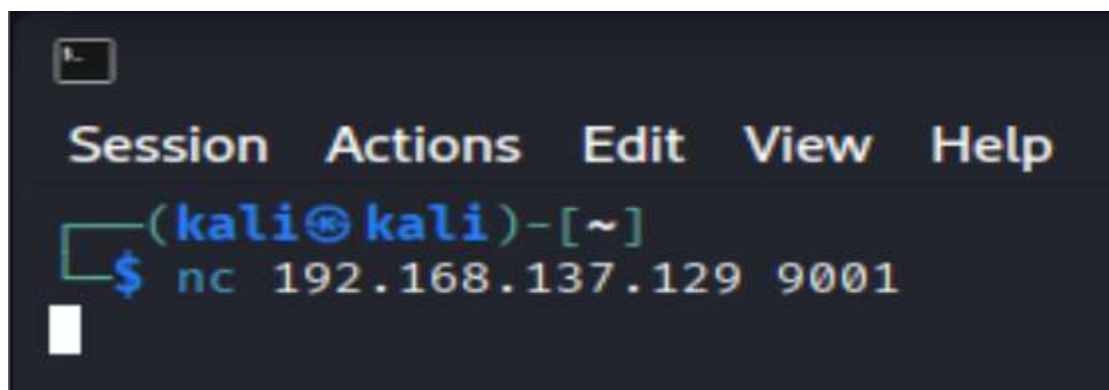
the last step for ModSecurity to block the attacks. Finally, with the command "**sudo**

service apache2 restart" we restart the web server to apply all the changes we made to the settings file and for the system to read them so that the WAF can be activated.

Now we will go to the client (Kali Linux) and use the Ubuntu ip (server) to check port 80. We will do this by writing the ip in the client's browser.



Since it shows us this page, it means that everything is working as it should. Then we will check port 9001 with the Netcat tool. We will open a terminal in Kali Linux and write the command "**nc 192.168.137.129 9001**". We notice that the connection is successful if the cursor flashes.



To confirm this we will write the message "HELLO" just below and press Enter. We will temporarily return to Ubuntu and go to the terminal we had left open.

```

Setting up libapache2-mod-security2 (2.9.7-1build3) ...
apache2_invoke: Enable module security2
Processing triggers for libc-bin (2.39-0ubuntu8.6) ...
Processing triggers for ufw (0.36.2-6) ...
Processing triggers for man-db (2.12.0-4build2) ...
kalaitron@ubuntudesktop1:~$ sudo service apache2 start
kalaitron@ubuntudesktop1:~$ sudo service apache2 status
● apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
   Active: active (running) since Wed 2025-12-24 08:55:14 EET; 1min 49s ago
     Docs: https://httpd.apache.org/docs/2.4/
   Process: 6385 ExecStart=/usr/sbin/apachectl start (code=exited, status=0/SUCCESS)
 Main PID: 6389 (apache2)
    Tasks: 55 (limit: 9374)
   Memory: 29.8M (peak: 29.6M)
      CPU: 499ms
   CGroup: /system.slice/apache2.service
           └─ 6389 /usr/sbin/apache2 -k start
             6394 /usr/sbin/apache2 -k start
             6396 /usr/sbin/apache2 -k start

Dec 24 08:55:13 ubuntudesktop1.kalaitron systemd[1]: Starting apache2.service - The Apache HTTP Server...
Dec 24 08:55:14 ubuntudesktop1.kalaitron systemd[1]: Started apache2.service - The Apache HTTP Server.
kalaitron@ubuntudesktop1:~$ q
Command 'q' not found, but can be installed with:
sudo apt install q # version 1.6.3-1, or
sudo apt install python3-q-text-as-data # version 3.1.6-3
See 'snap info q' for additional versions.
kalaitron@ubuntudesktop1:~$ nc -l -p 9001
HELLO

```

We note that the message "HELLO" appeared here as well. This is proof that communication works. In other words, the client managed to successfully connect to Ubuntu port 9001 (server). So the iptables rule we made earlier works properly. If the firewall was set up incorrectly, this message would never arrive and would not appear in the server terminal.

```

(kali@kali)-[~]
$ nmap -p 8080 192.168.137.129
Starting Nmap 7.95 ( https://nmap.org ) at 2025-12-23 19:01 EST
Nmap scan report for 192.168.137.129
Host is up (0.0018s latency).

PORT      STATE SERVICE
8080/tcp  filtered http-proxy
MAC Address: 00:0C:29:F5:51:1F (VMware)

Nmap done: 1 IP address (1 host up) scanned in 0.59 seconds

(kali@kali)-[~]
$

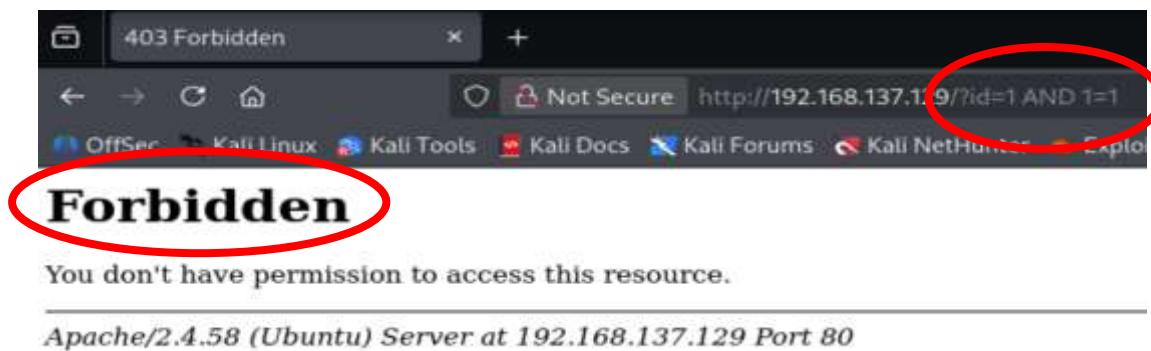
```

Finally with the command "nmap -p 8080 192.168.137.129» we check port 8080 in Ubuntu from Kali Linux with the Nmap tool. The result, as we observe, is that the firewall cut (DROP) and did not send any response back. So iptables on Ubuntu did

its

job

properly.



To confirm, we added the parameter "`?id=1 AND 1=1`" as a simulation of a SQL Injection attack on the web server. As we can see, the web server returned an error message "**403 Forbidden**". Instead of displaying the regular page, the system refused to serve the request. This result proves that ModSecurity works properly as a Web Application Firewall (WAF). It detected the malicious pattern (`AND 1=1`) within the HTTP request and blocked it immediately, protecting the application from attack as required by the security script.

Security Policy for Home Office

To ensure a high level of safety in a home work environment (Home Office), it is recommended to adopt and implement the following practices:

- **Router Hardening:** The security of the router is enhanced by:
 - replacing default passwords with strong and unique ones;
 - the disabling of remote management,
 - disabling the UPnP protocol, which can create uncontrolled door openings.
- **Device and Update Management:** It is necessary to enable automatic security updates for the operating system and installed applications. At the same time, every work device must have protection software (Antivirus or EDR) as well as an active local firewall (host-based firewall), in order to reduce the attack surface.
- **Use of Virtual Private Network (VPN):** Access to corporate resources and services should be carried out exclusively through a VPN, which provides encrypted communication. Using a VPN ensures both the confidentiality and

integrity of the data transferred, protecting it from interception or sniffing attacks.

- **Network Segmentation:** The home network should be divided into at least two distinct zones, via VLANs or the router's "Guest Network" function. Devices used for business purposes must operate on an isolated network, cut off from untrusted or uncontrolled devices (e.g. IoT devices, guest devices or game consoles). This significantly reduces the possibility of lateral disruption lateral movement in the event of a device being tampered with.
- **Physical Workplace Security:** The physical protection of information is ensured by automatically or manually locking the screen when leaving the workplace and applying a "clean desk" policy, so that sensitive documents or information are not accessible to third parties.

Bibliography

- Acunetix. (n.d.). 10 Tips to Secure Your Apache Web Server. <https://www.acunetix.com/blog/articles/10-tips-secure-apache-installation/>
- Apache Software Foundation. (n.d.). Apache HTTP Server Version 2.4 Documentation. <https://httpd.apache.org/docs/2.4/>
- Apache Software Foundation. (n.d.). Security Tips for Apache Server. https://httpd.apache.org/docs/2.4/misc/security_tips.html
- Cloudflare. (n.d.). What is a WAF? | Web Application Firewall explained. <https://www.cloudflare.com/learning/ddos/glossary/web-application-firewall-waf/>

- Computerphile. (2016, May 11). SQL Injection Explained [Video file]. <https://www.youtube.com/watch?v=ciNHn38EyRc>
- DigitalOcean. (2023). Iptables Essentials: Common Firewall Rules and Commands. <https://www.digitalocean.com/community/tutorials/ufw-essentials-common-firewall-rules-and-commands>
- GeeksforGeeks. (2024). iptables command in Linux with Examples. <https://www.geeksforgeeks.org/linux-unix/iptables-command-in-linux-with-examples/>
- Hussein Nasser. (2020, September 20). Apache ModSecurity Tutorial [Video file]. <https://www.youtube.com/watch?v=MB7nQrIP5Yc>
- Linux Manual. (n.d.). Ncat - Netcat for the 21st Century. <https://man7.org/linux/man-pages/man1/ncat.1.html>
- Netfilter Project. (n.d.). iptables(8)-Linux man page. <https://linux.die.net/man/8/iptables>
- NetworkChuck. (2021, July 15). Linux Firewalls (iptables) for Beginners. <https://www.youtube.com/watch?v=v-z6LCcrutw>
- OWASP Foundation. (n.d.). OWASP ModSecurity Core Rule Set (CRS). <https://owasp.org/www-project-modsecurity-core-rule-set/>
- OWASP Foundation. (n.d.). SQL Injection. https://owasp.org/www-community/attacks/SQL_Injection
- Palo Alto Networks. (n.d.). What is a Web Application Firewall (WAF)?. <https://www.paloaltonetworks.com/cyberpedia/what-is-a-web-application-firewall>
- SpiderLabs. (n.d.). ModSecurity Reference Manual. <https://github.com/SpiderLabs/ModSecurity/wiki>
- TecMint. (2023). How to Install ModSecurity with Apache on Debian/Ubuntu. <https://www.tecmint.com/install-modsecurity-apache-debian-ubuntu/>